GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

vLLM项目最新特性和未来展望

Jiangyun Zhu

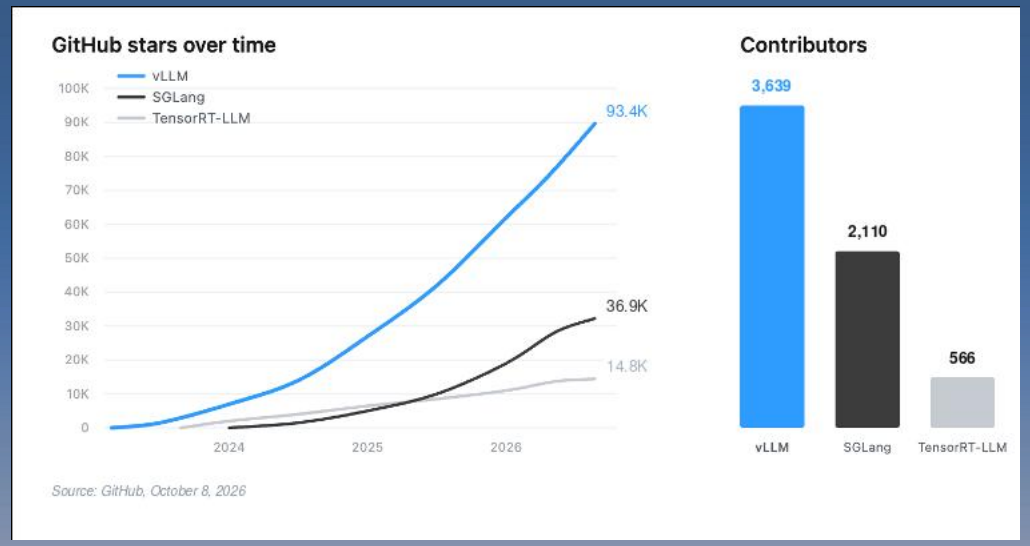


GOSIM

GOSIM Shenzhen 2026

The world's leading open inference engine

COMMUNITY



vLLM is the most used inference engine.

ECOSYSTEM

APPLICATION LAYER

Meta · aws · salesforce · Uber · Roblox · ByteDance · databricks

FRONTIER LABS

Kimi · MINIMAX · THINKING MACHINES · deepseek · Qwen · Mistral AI

INFERENCE CLOUD

baseten · Fireworks AI · CoreWeave · Crusoe · Amazon Bedrock

ECOSYSTEM PARTNERS

NVIDIA · AMD · Google · Red Hat · PyTorch

vLLM is the AI ecosystem default.

GOSIM

GOSIM Shenzhen 2026

vLLM's vantage point comes is at the center of these.

Hardware

whose silicon,
at what cost

×

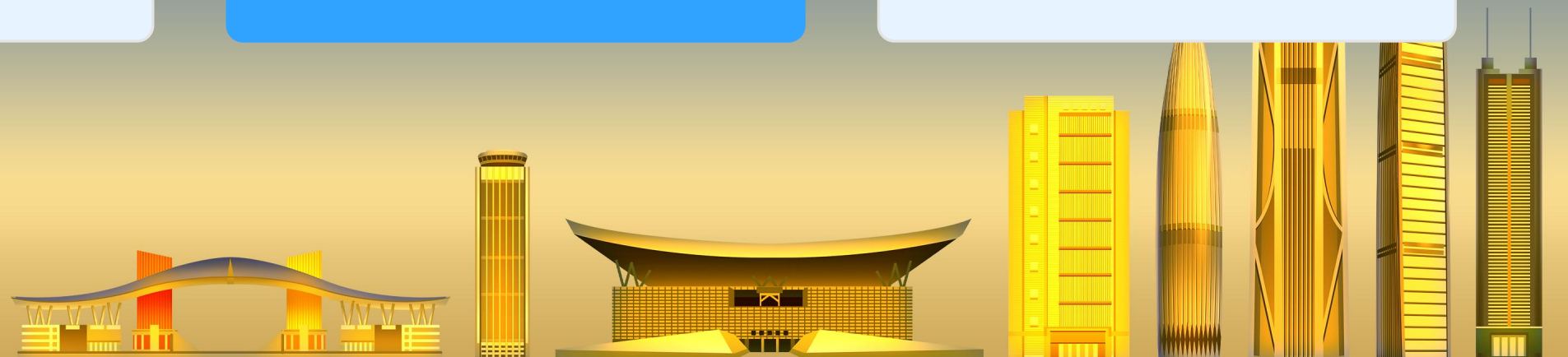
Workloads

chat, or agentic
& specialized

×

Models

which architectures,
and how many



CONTENTS

Part 01 New Models

Part 02 More Aggressive Kernel Fusion

Part 03 Core Re-design and Update

Part 04 KV cache management

Part 05 New Parallelism

Part 04 Future

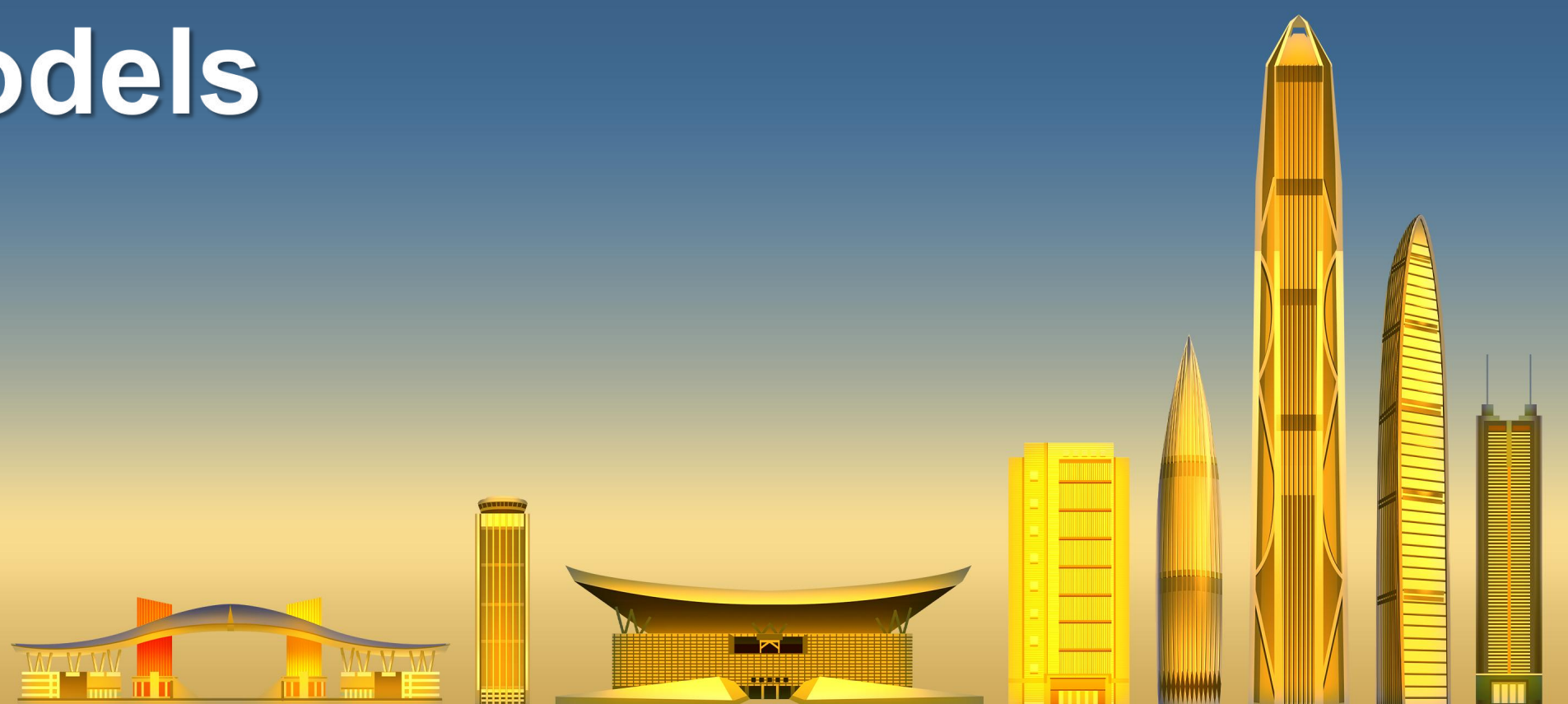


GOSIM

GOSIM Shenzhen 2026

0 New models

1



Models are becoming more diverse

Size, architecture, modality are all growing in variance per model

MODEL	SIZE	ATTENTION
Kimi K3	2.8T MoE	KDA
GLM 5.3 Flash	320B MoE	KDA, Sparse
DeepSeek V4.1 Flash	552B MoE	CSA2, CED
Qwen3.8 Flash Next	125B	QSA, GDN
MiniMax M3	456B MoE	MSA

A snapshot — the open-weight frontier, today.

- Day-0 support for every new frontier model
- Multiple parallelism strategies — tensor, pipeline, expert, data — combined to fit any MoE
- A dedicated kernel per variant — MLA, sparse (DSA), gated-delta, linear
- Prefix caching for long context



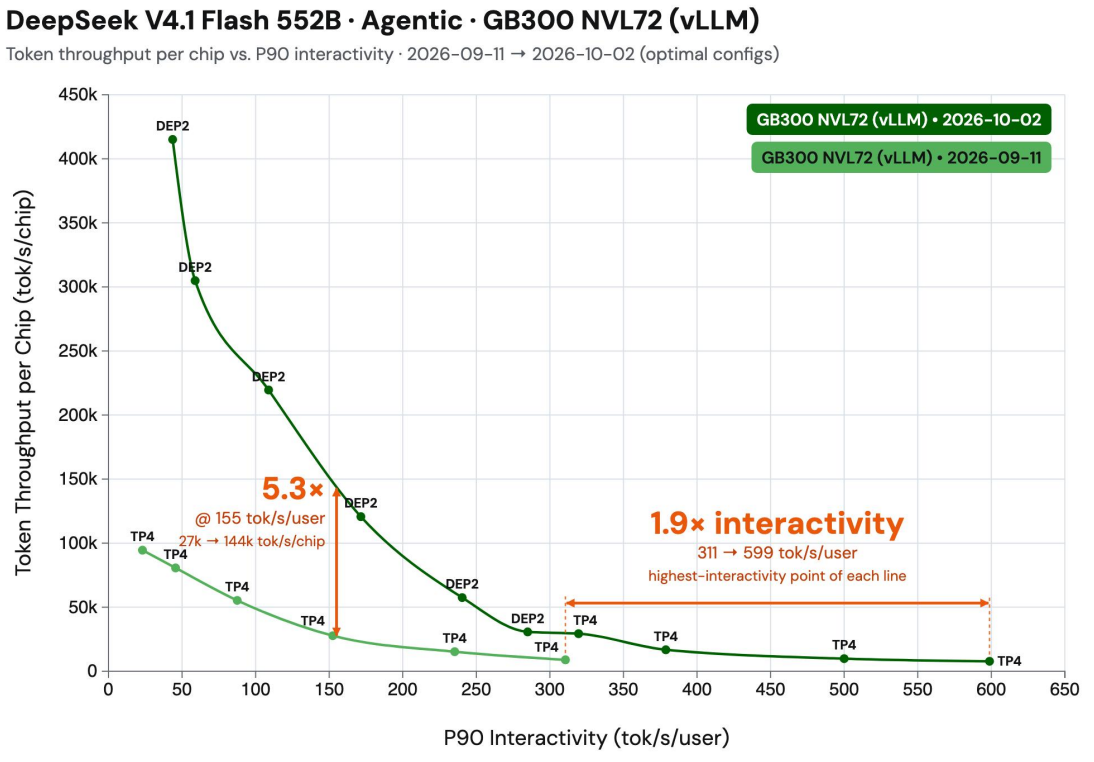
Day0 support is just the beginning

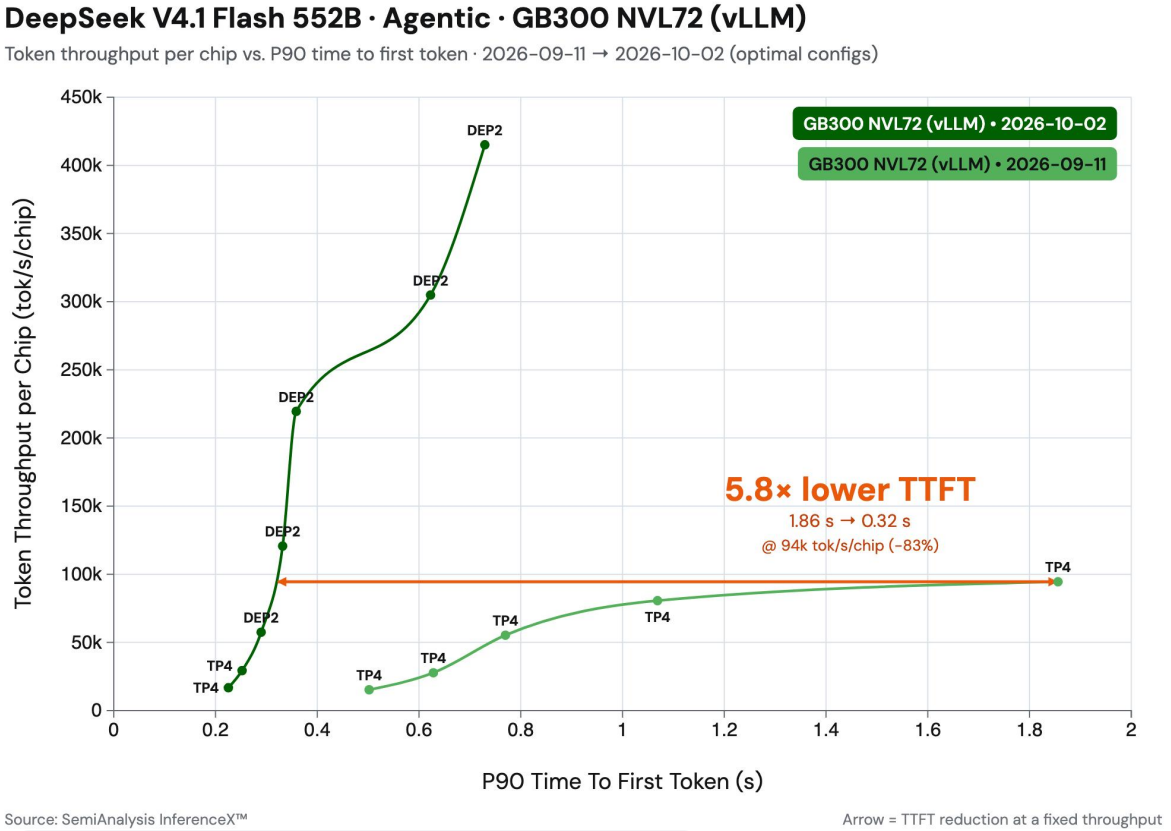


https://vllm.ai/blog/2026-09-13-kimi-k3-performance-optimization

Day0 support is just the beginning

https://vllm.ai/blog/2026-10-07-deepseek-v41-flash





4.4× 最高吞吐
95K → 415K（读图）

5.3× 同交互速度吞吐
@155 tok/s/user

1.9× 最快交互速度
311 → 599 tok/s/user

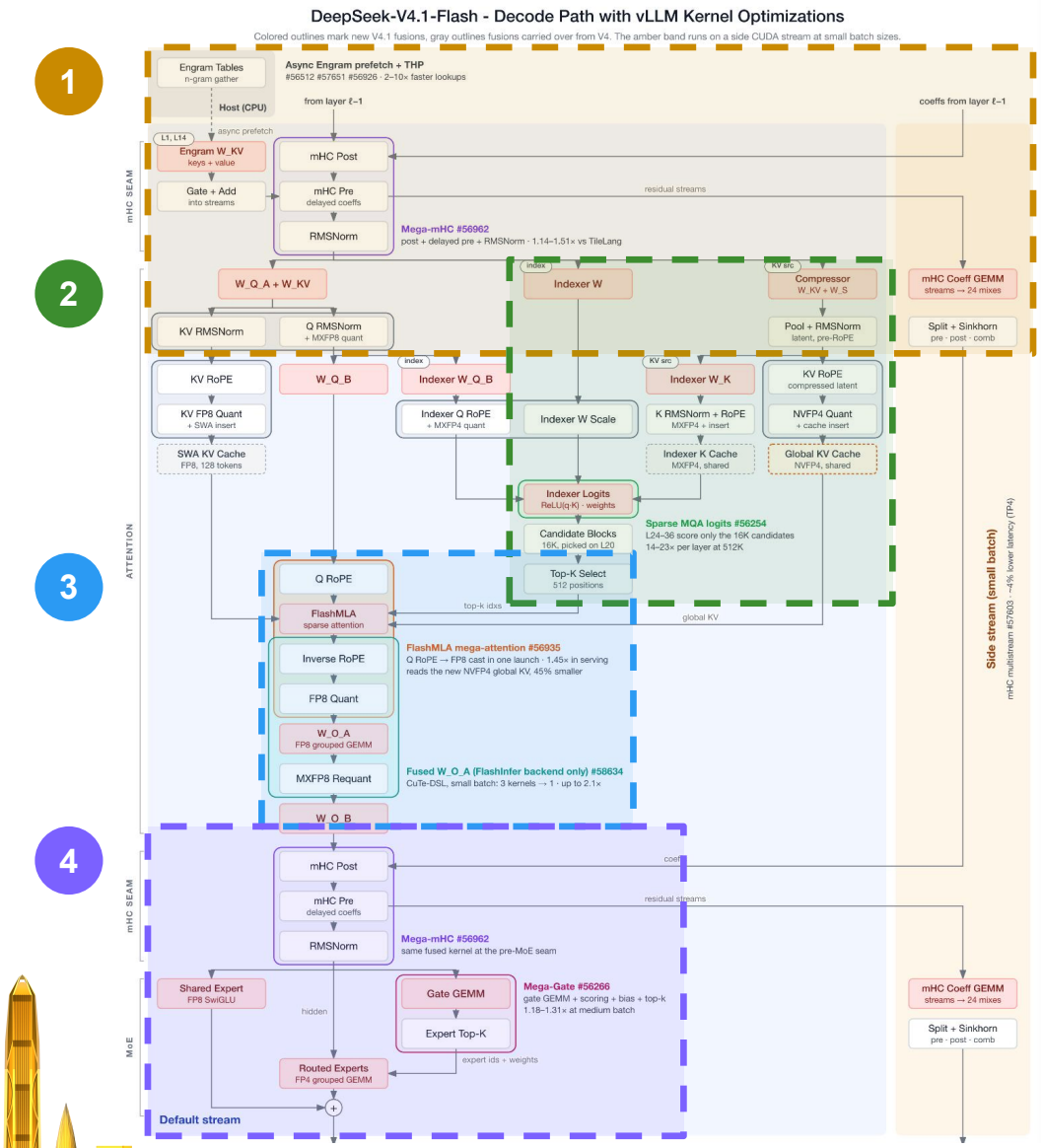
5.8× 更低的 P90 TTFT
1.86 → 0.32 s

0

More aggressive kernel fusion

2





一层 decode 的数据流：四段

vLLM blog：DeepSeek-V4.1-Flash decode path

1 mHC seam

Engram (L1 / L14) → Post → Pre → RMSNorm

2 indexer

打分 → 候选池 (L20) → top-512

3 attention

Q RoPE → FlashMLA (稀疏) → W_O_A → W_O_B

4 mHC seam + MoE

mHC seam → Gate (top-6) → 专家 (384 + 1)

右侧橙色带：小 batch 时，下一段的 mHC 系数在 side stream 上算

Kernel：四段各做了哪些优化

① mHC seam

写回 4 路残差 → 合成 1 路

原来：Post | 算系数 | Pre | Norm

现在：**Mega-mHC**

1.14–1.51×

deepseek-ai/**DeepGEMM**

post + pre + RMSNorm

- **side stream** 算系数和 attention 并行 **−4%**
- **all-reduce 融合** TP 通信融进 mHC **−9%**
- **Engram** 预取 + 大页，查表 **10×**

② indexer

给全局 KV 打分，挑 top-512

原来：整个上下文都打分，再 mask

现在：**只打 16K 候选** **sparse MQA logits**

14–23×

deepseek-ai/**DeepGEMM + DeepSelect**

打分 + top-k · 512K 时 · 默认关闭

长上下文 decode 收益最大

③ attention

640 条 KV（128 SWA + 512）

原来：RoPE | attn | inv RoPE | FP8

现在：**mega-attention**

1.45×

deepseek-ai/**FlashMLA**

直接读 FP4 全局 KV

- **fused W_O_A** CuTe DSL：3 → 1 · ITL **−6~7%**

④ MoE

前面同样一段 mHC seam

原来：GEMM | 打分 | bias | top-k

现在：**Mega-Gate**

1.18–1.31×

deepseek-ai/**DeepGEMM**

gate + 打分 + top-k

MoE 前的 mHC seam 同 ①

集成 DeepSeek 开源 kernel　　vLLM 自己的融合 / 并行

Lesson we learned: vLLM x torch.compile

- **Automatic kernel generation**
  - Fast iteration, automatic fusion, performance portability
  - RoPE, RMSNorm, SiluMul, QuantFP8, etc.
- **Applying optimizations independently of model definitions**
  - Custom passes (mostly fusion)
  - Piecewise CUDAGraph
  - Other graph-level transformations (Nanoflow, DBO)



Example

```python
from torch.fx import subgraph_rewriter
# This is an inplace operation.
def replace_patterns(graph_module):
    """
    Original:
        def f(x, y):
            x = x + y
            x = x * y
            return x

    After pass:
        def f(x, y):
            return x - y
    """
    def pattern(x, y):
        x = torch.ops.aten.add.Tensor(x, y)
        x = torch.ops.aten.mul.Tensor(x, y)
        return x

    def replacement(x, y):
        return torch.ops.aten.sub.Tensor(x, y)

replaced_patterns = subgraph_rewriter.replace_pattern_with_filters(
    traced_module, pattern, replacement
)
```



Example

```python
def pattern(q: torch.Tensor, k: torch.Tensor, v: torch.Tensor,
            output_attn: torch.Tensor, output_quant: torch.Tensor,
            scale: torch.Tensor):
    at1 = auto_functionalized(torch.ops.vllm.unified_attention_with_output,
                              query=q,
                              key=k,
                              value=v,
                              output=output_attn,
                              layer_name=self.layer_name,
                              output_scale=None)
    attn_out_v = torch.ops.aten.reshape(at1[1], [-1, num_heads * head_size])

    at2 = auto_functionalized(torch.ops._C.static_scaled_fp8_quant,
                              result=output_quant,
                              input=attn_out_v,
                              scale=scale)
    return at2[1]

def replacement(q: torch.Tensor, k: torch.Tensor, v: torch.Tensor,
                output_attn: torch.Tensor, output_quant: torch.Tensor,
                scale: torch.Tensor):
    at1 = auto_functionalized(torch.ops.vllm.unified_attention_with_output,
                              query=q,
                              key=k,
                              value=v,
                              output=output_quant,
                              layer_name=self.layer_name,
                              output_scale=scale)
    return torch.ops.aten.reshape(at1[1], [-1, num_heads * head_size])
```

```python
class LlamaAttention:
  qkv_proj: Linear
  attention: Attention
  o_proj: Linear
```

Attention layer definition
(simplified)

Attention + QuantFP8 fusion

- Fuse o_proj input quantization onto attention
- Use Inductor pattern matcher, operating on the fx.Graph
- No impact to model definition or Linear/Attention abstractions

But things have changed now

Slow Startup

Torch compile is very slow in startup time

Hard to Maintain

Complex compiler backends raise maintenance overhead

Rising Complexity

Modern model architectures are more and more complex

Suboptimal Performance

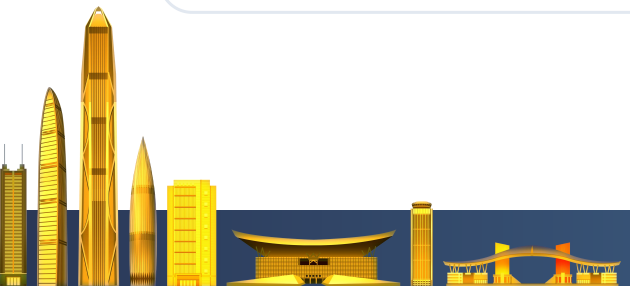
Performance is not ideal compared to speed-of-light

Opaque Kernels

The generated kernel is completely opaque to the agent

Capable Agents

The agent is highly proficient at programming now

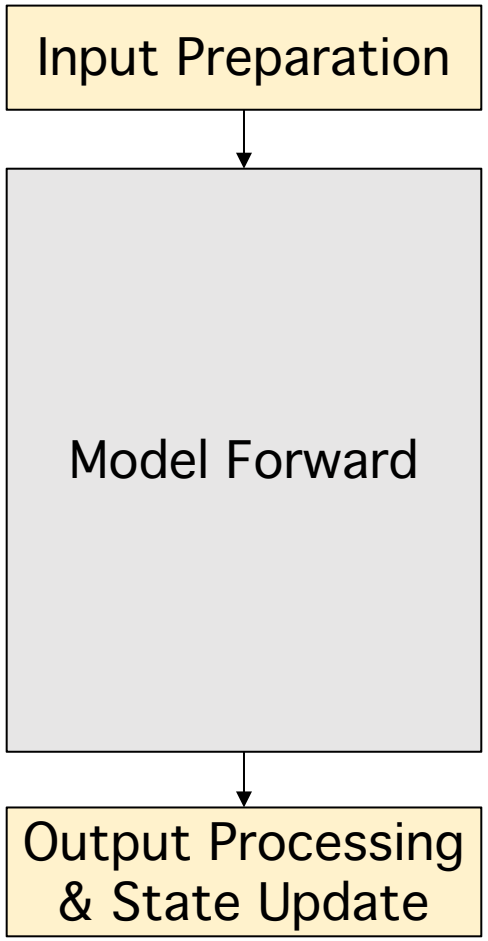


0 Core Re-design

3

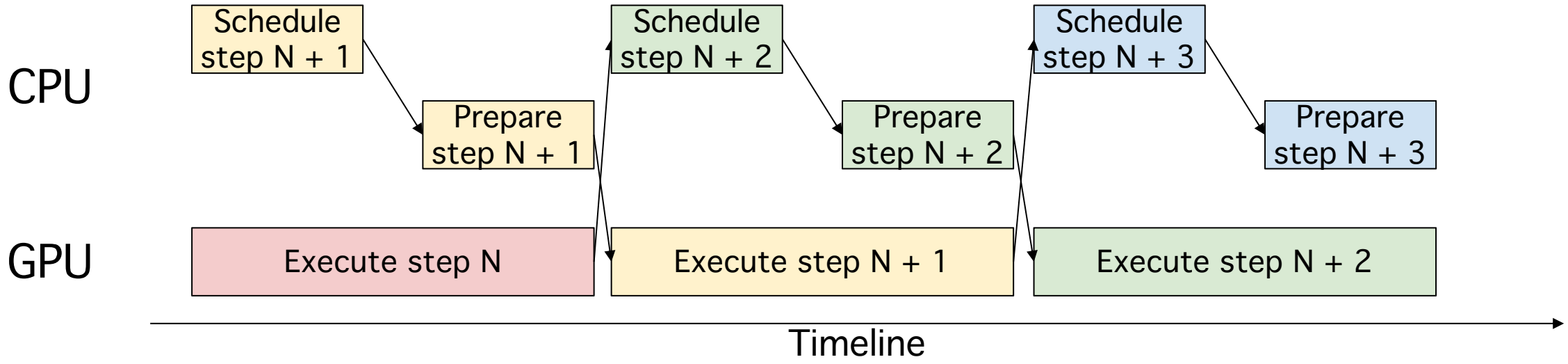


Core Re-design (1): Model Runner V2



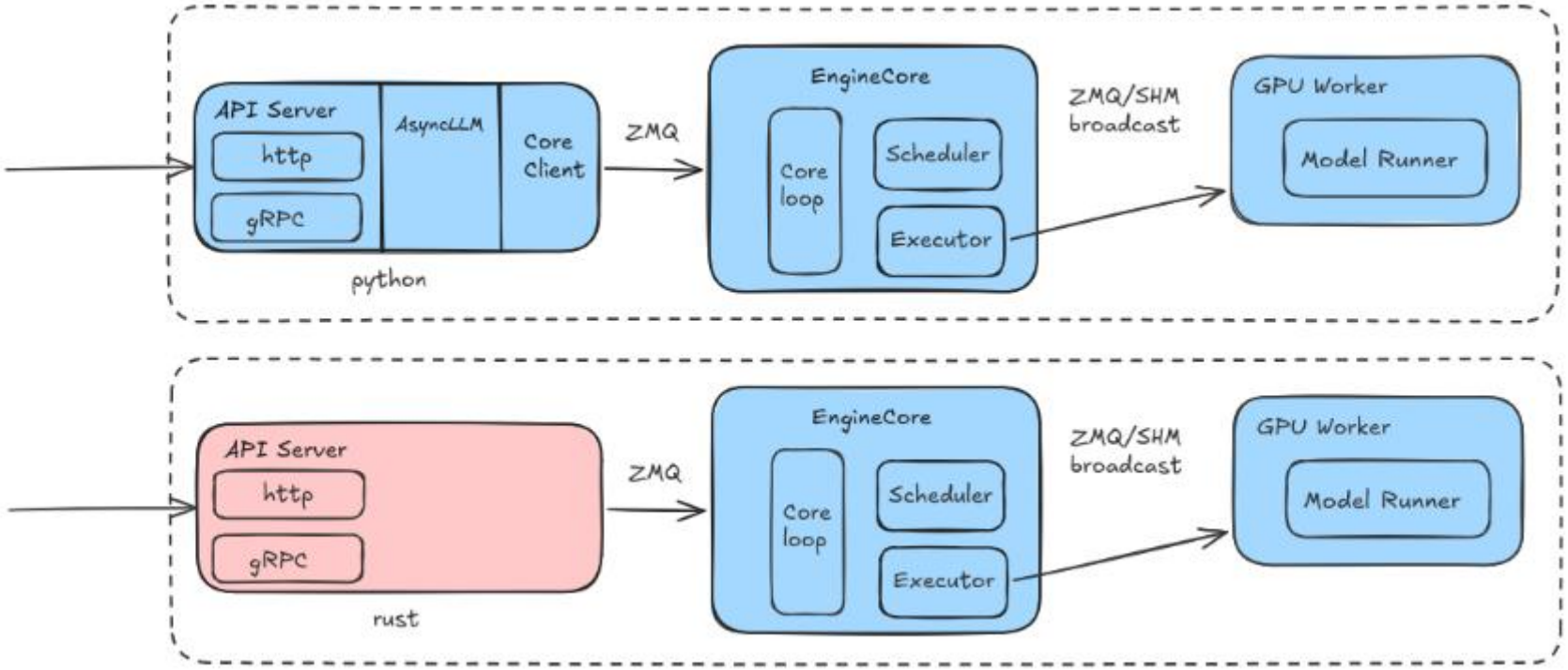
- Model runner is a component that performs complex bookkeeping for batching, paged attention, spec decoding, sampling, RL features, etc.
- MRV1 implemented the bookkeeping logic on CPU, which was slow and hacky
- MRV2 replaces this with custom Triton kernels, which is fast and clean

Model Runner V2: Async-first Design



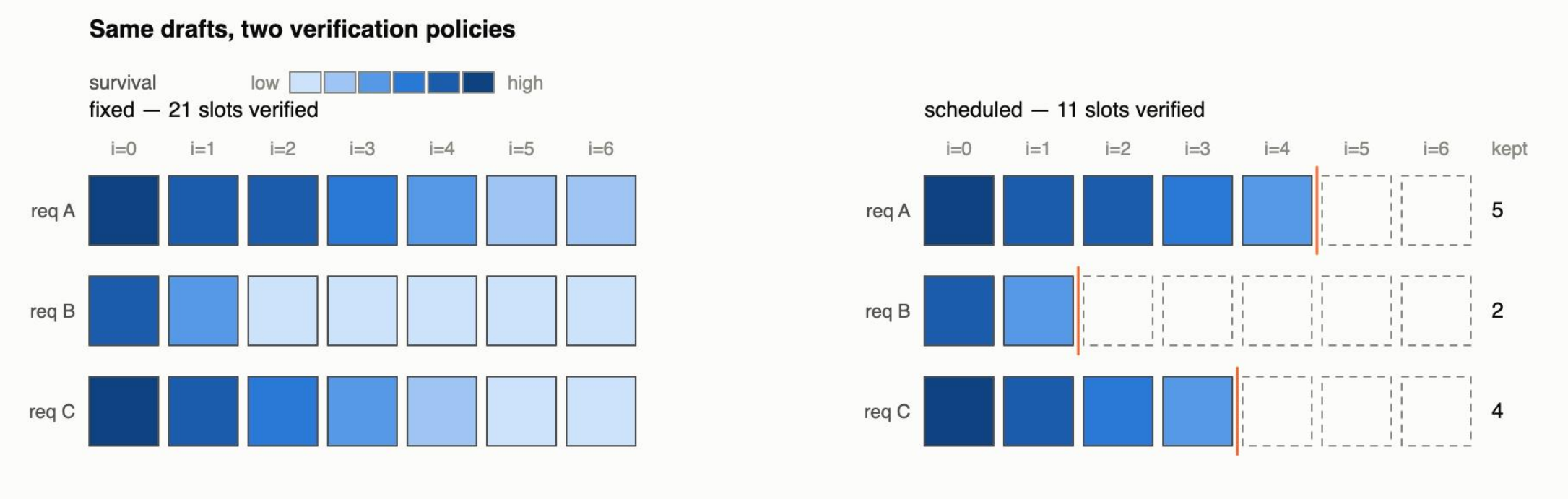
- vLLM uses *async scheduling* to overlap host overheads with model execution
  - Core idea: “schedule & prepare the next batch one step ahead”
- MRV2 ensures *zero synchronization*, so the GPU is never stalled by the CPU in *any cases* including speculative decoding

Core Re-design (2): Rust Frontend



- Stable performance (P90 TTFT & TPOT)
- Simple scale-out
- Shared crates with other Rust-based frameworks

Core Update: adaptive verification



https://vllm.ai/blog/2026-08-14-dspark-adaptive-verification

GOSIM

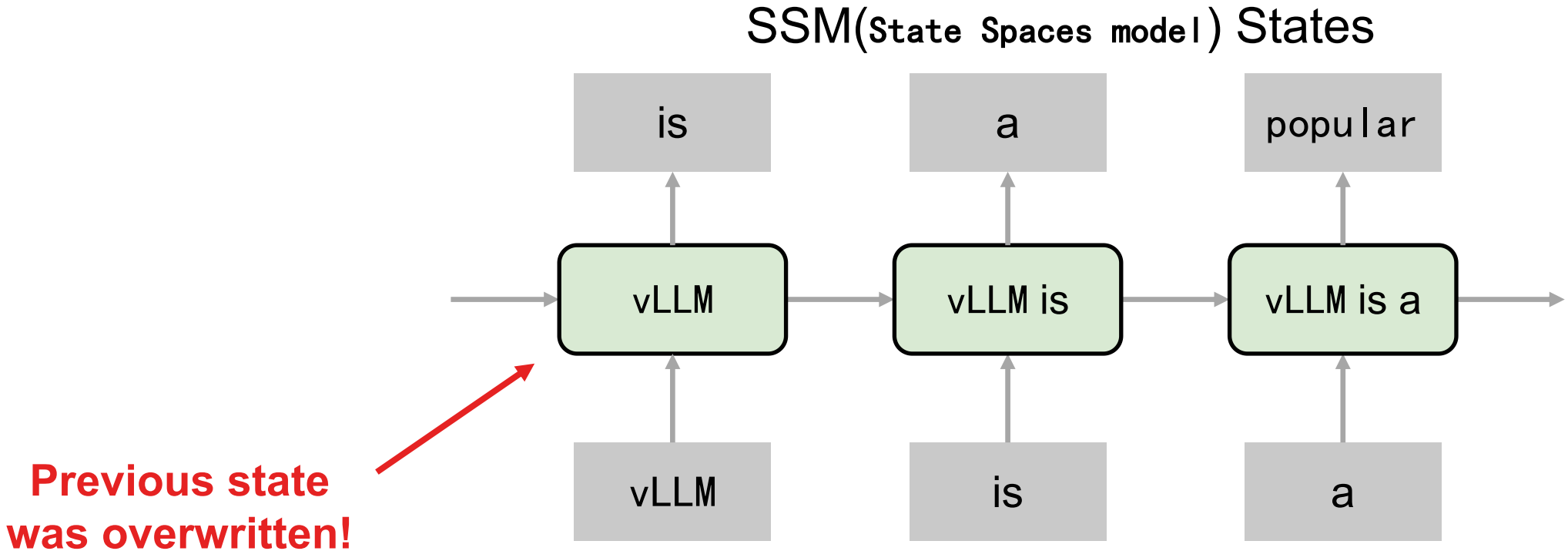
GOSIM Shenzhen 2026

0

KV Cache Management

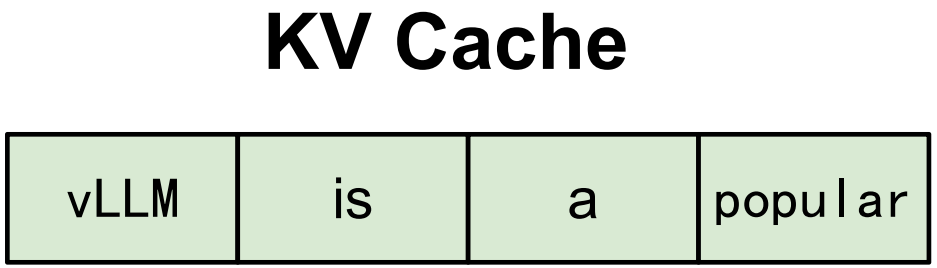
4



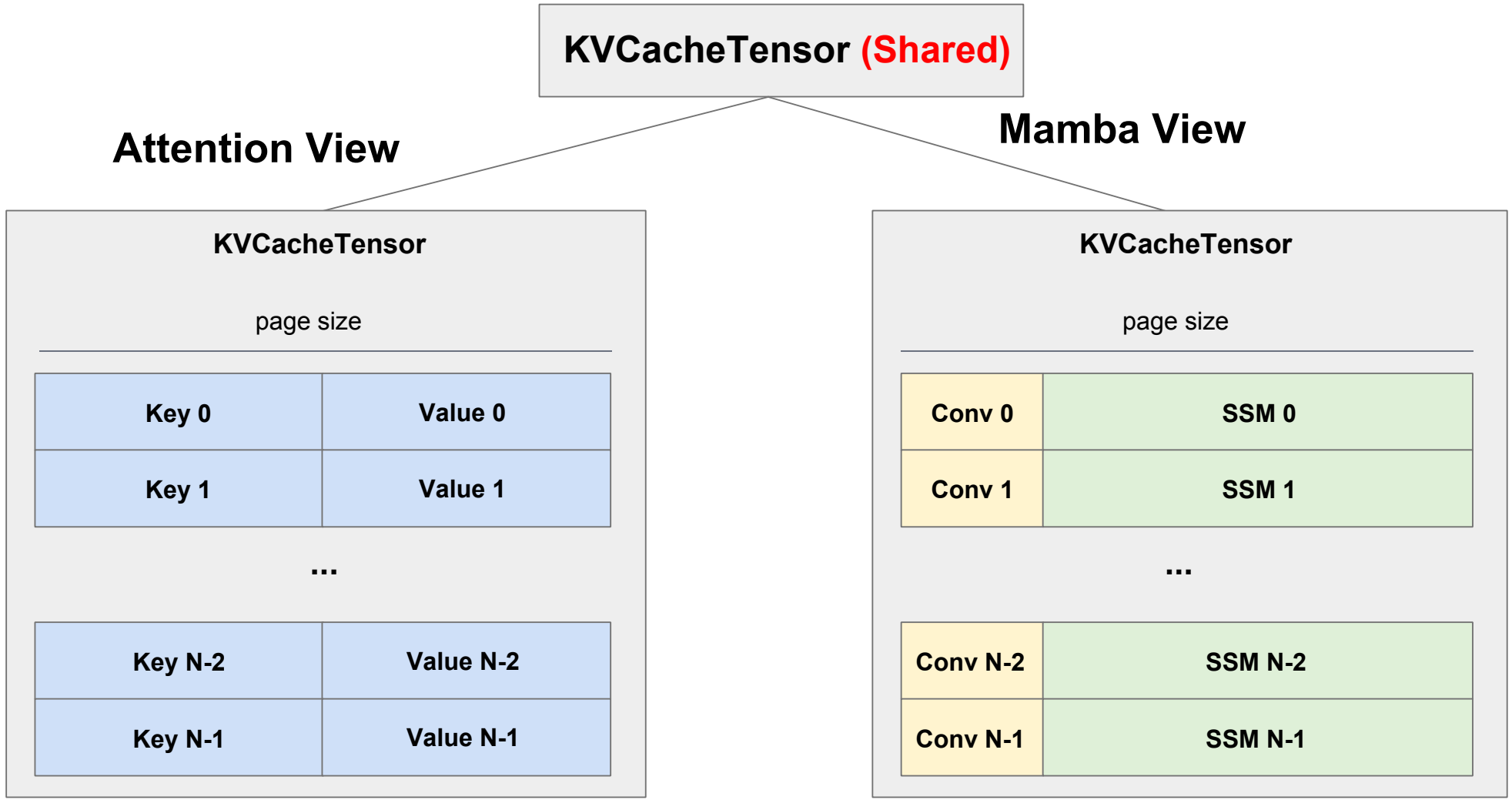


SSM's modeling win complicates their systems win!

- This breaks the assumption of vLLM: the kv cache grows continuously and is append-only.
- How to do:
  - memory management for hybrid cache
  - prefix caching



How to mange: Hybrid Memory Allocator



Hybrid Memory Allocator

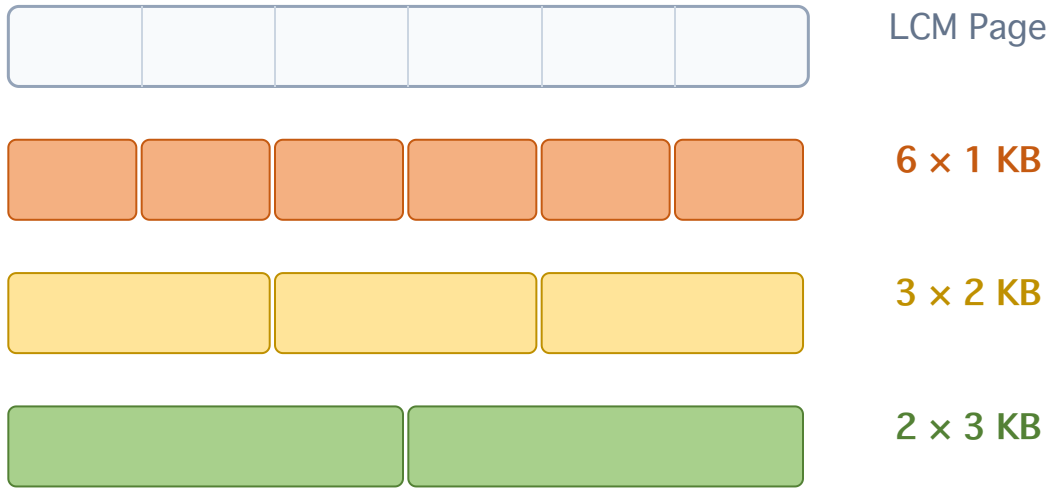
- Split the memory into pages whose size is the Least Common Multiple (LCM) of all possible allocation sizes
- For each type, further split one LCM page into *multiple small pages* of specific size with no fragmentation

Data Type 1 (full attn)
Size X = 1 KB

Data Type 2 (sliding window)
Size Y = 2 KB

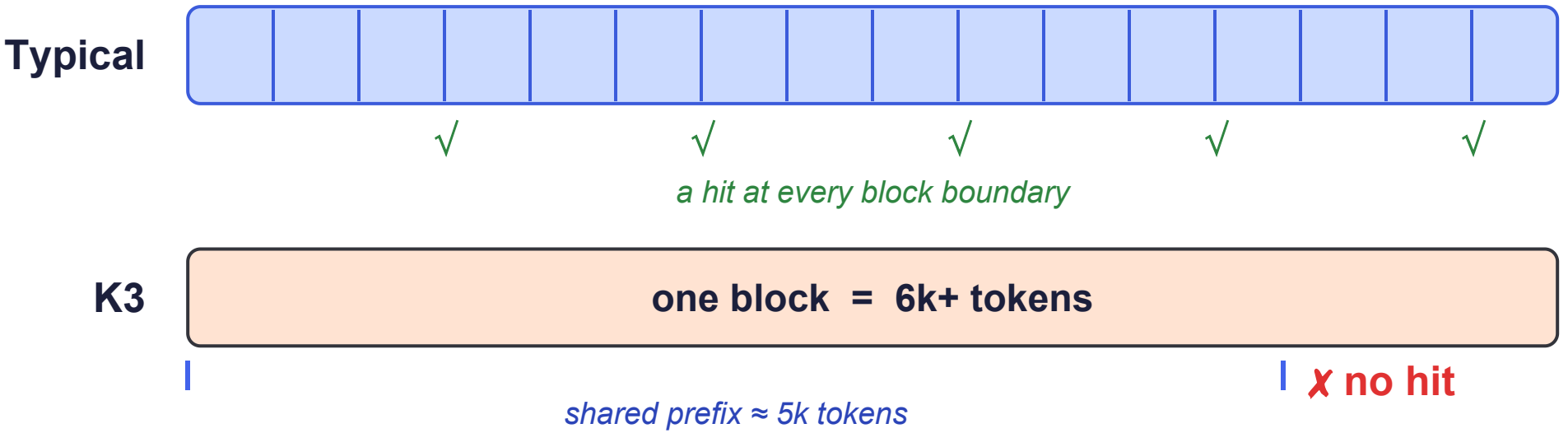
Data Type 3 (mamba)
Size Z = 3 KB

LCM Page (Size = LCM(X, Y, Z) = 6 KB)

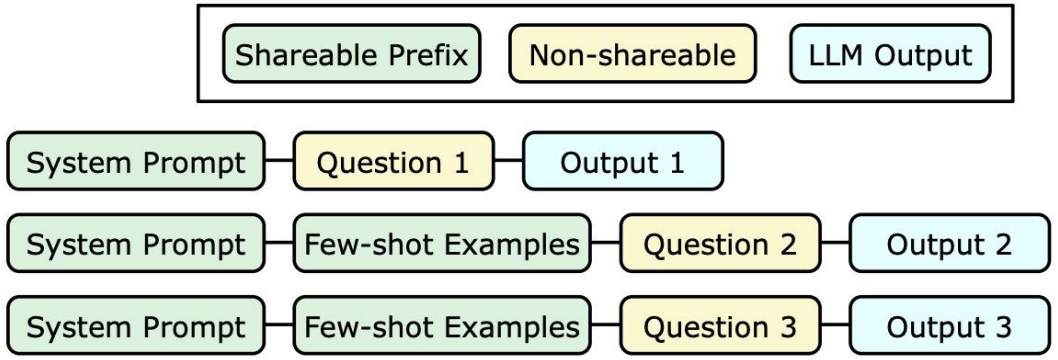


Weakness of existing design

- We need to align the page sizes across different KV cache groups.
- As a result, if one cache is much larger than another, the smaller cache may require a very large block size to match it.
- K3: KDA state is about 600x larger than MLA cache, the block size of K3 is 6k+ in DP case, this makes prefix cache very inefficient

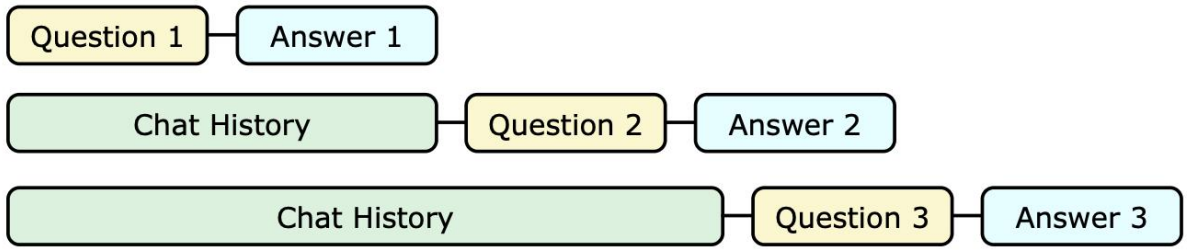


What's the most valuable cache



(a) System prompt and few-shot prompting

System Prompt



(b) Multi-turn conversation (e.g., ChatGPT)

The boundary of each turn

Partial Cache Hit: Decouple Cache-Hit Granularity from Block Size

vLLM RFC #45702 (proposed by Moonshot) — introduce `hash_block_size` to hit a partial cache block

Example: block_size = 6, hash_block_size = 2

Request A prompt = a b c d e f g h (written to cache; registers both kinds of hash keys)



`H0 = hash([a b c d e f])` full-block key

`P0 = hash(parent=H0, [g h])` partial-tail key (at most one, for the final tail of a request)

Request B (before) prompt = a b c d e f g h x y — full-block (block_size) matching only



Recompute 4
6 tokens reused

✗ Second full-block key (H1) not cached ⇒ only block 0 is reusable; g h x y all recomputed from token 6

Request B (proposed) after the first full-block miss, probe partial-tail keys inside that block



Recompute 2
8 tokens reused

√ Hit (hit_length = 8) ⇒ allocate block_B, **Copy-on-Write** [g h], recompute only x y; H1 replaces P1 once the block fills

GOSIM

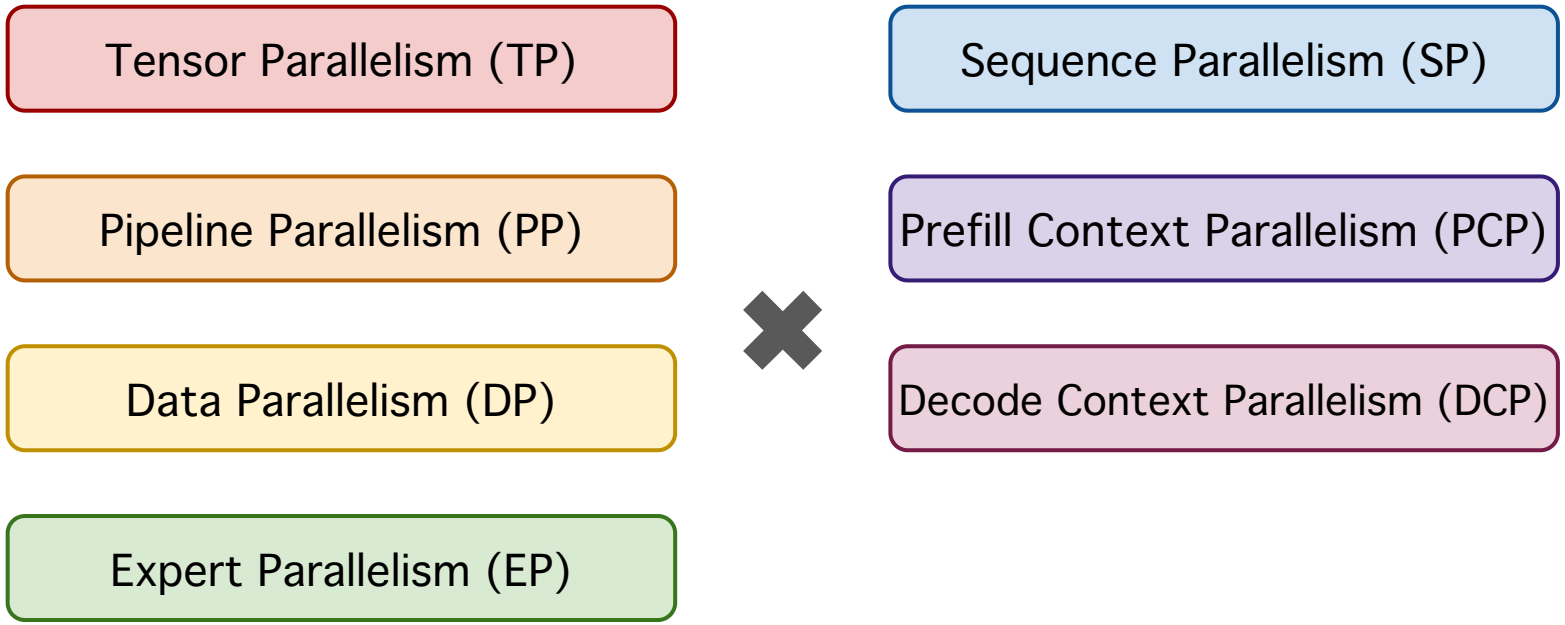
GOSIM Shenzhen 2026

New Parallelism

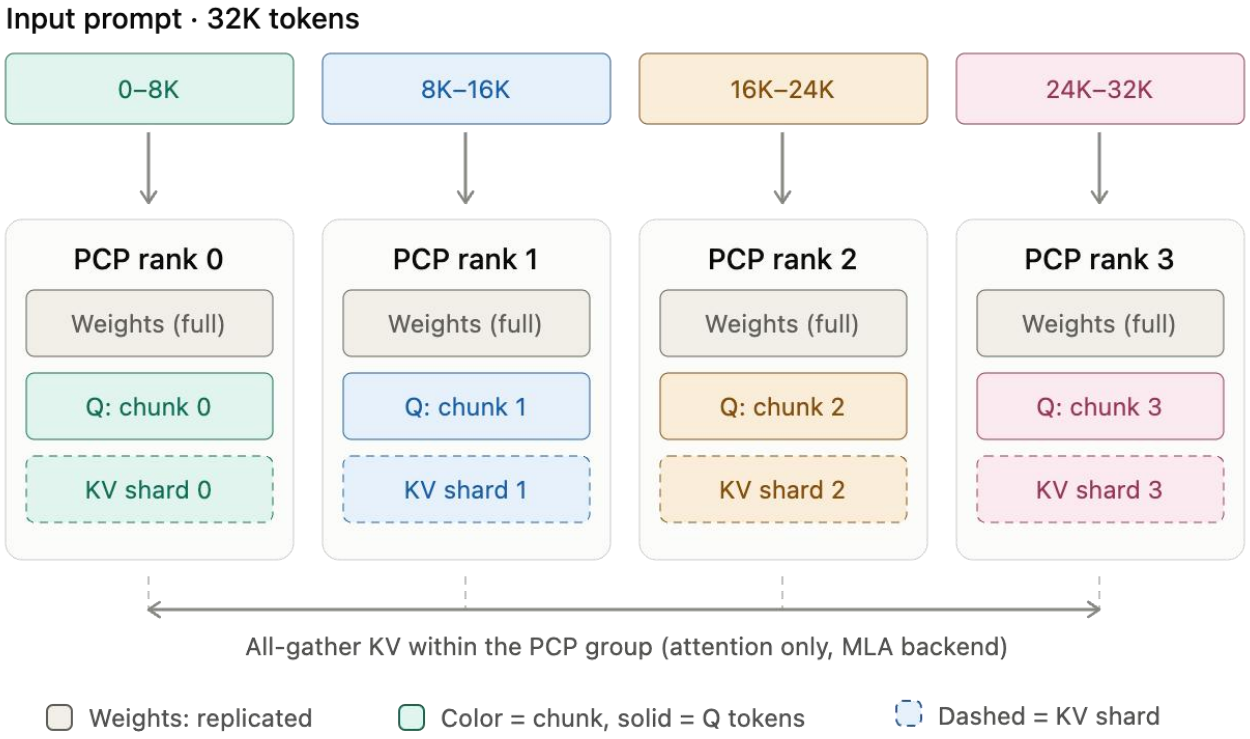


Model Parallelisms

- 2024: Llama architecture → TP
- 2025: DeepSeek V3 architecture → PD disagg + attention DP & MoE EP
- 2026: Kimi K3, GLM 5.3, DeepSeek V4, ... → PD disagg + ???



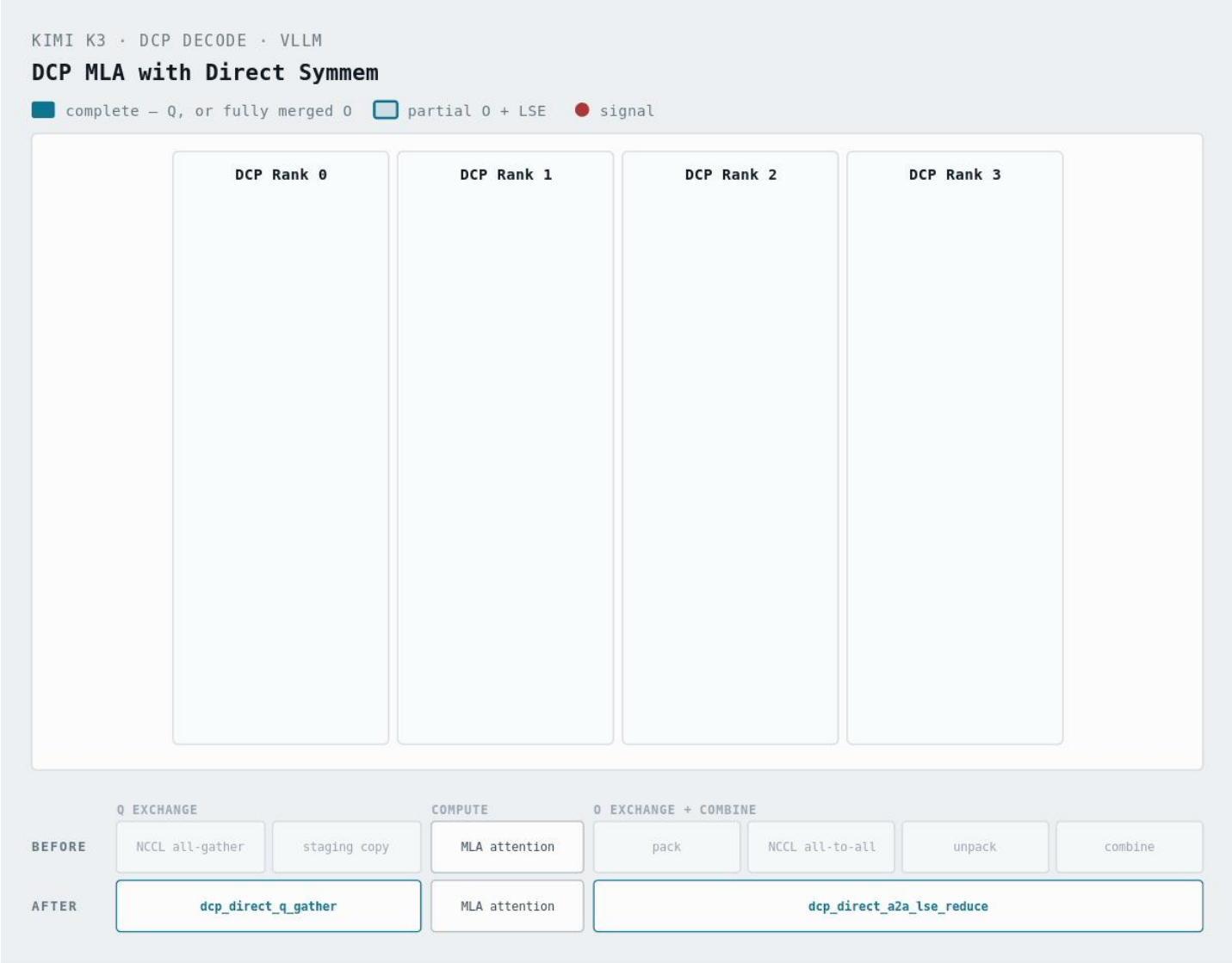
Case Study: GLM 5.3 + PCP



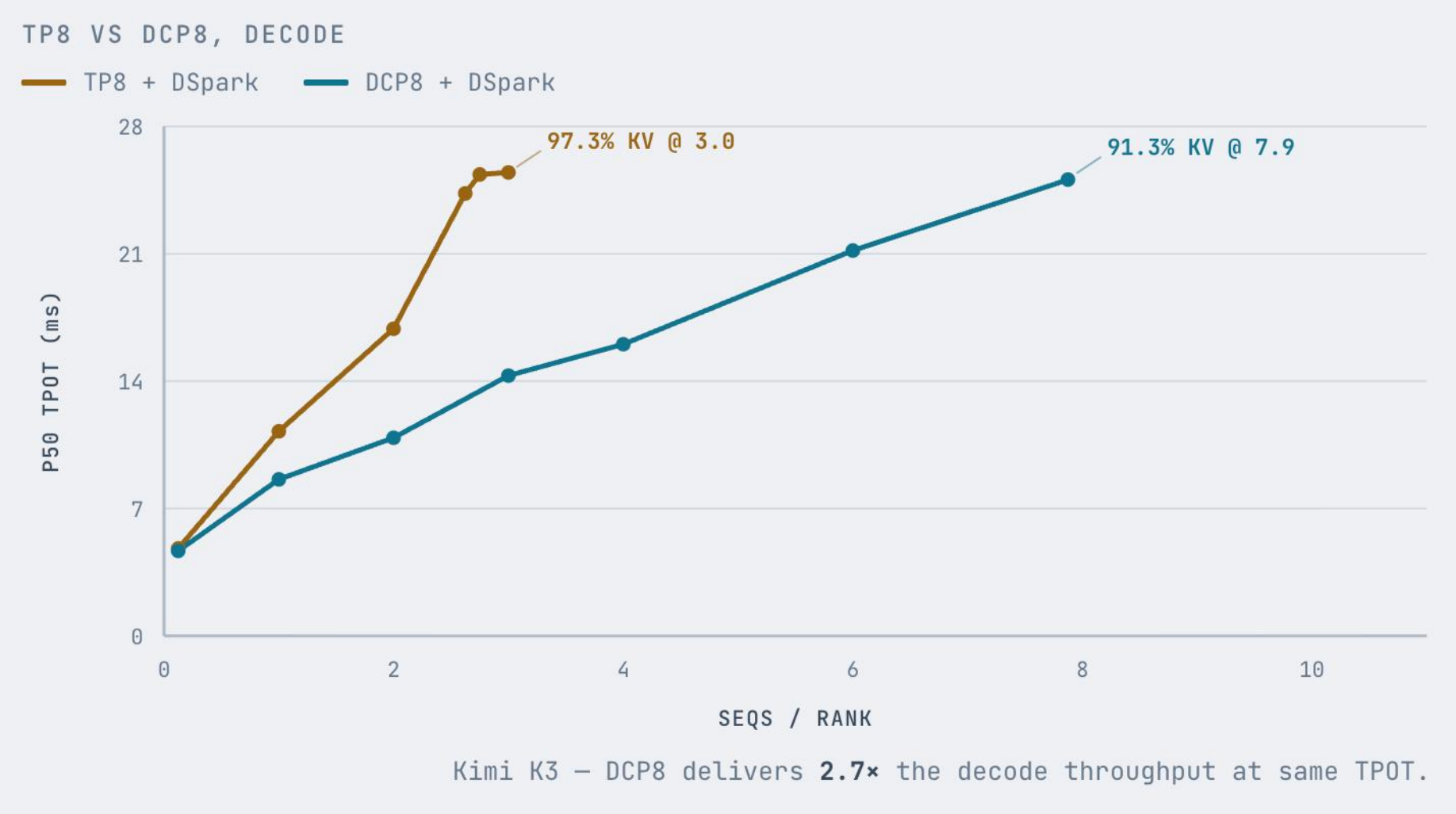
- TP performs very bad for GLM 5.3 prefill
  - Indexer overhead is duplicated
  - Sparse attn is memory-bound so sharding query heads doesn't help

- PCP performs the best for GLM 5.3 prefill
  - Indexer overhead is sharded
  - Sparse attn overhead is also sharded
  - 3.5x end-to-end speedup in TTFT compared to TP

Case Study: Kimi K3 DCP



Case Study: Kimi K3 DCP

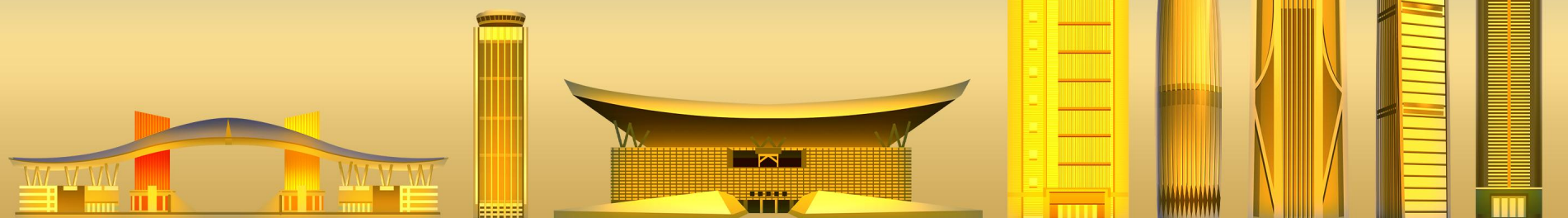


https://vllm.ai/blog/2026-09-08-vllm-agentx
https://vllm.ai/blog/2026-08-07-decode-context-parallelism

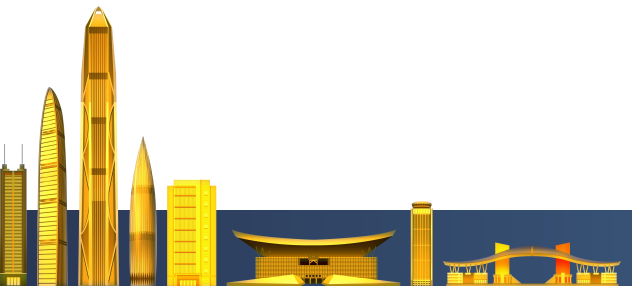
GOSIM

GOSIM Shenzhen 2026

Future



- How to move fast with ai
- KV cache manager & connector re-design
- More production feature
- Optimizations for broader hardware



GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM

GOSIM Shenzhen 2026

Thanks!

